

At a physical level, the data is stored in data files specific to the DBMS. Examples of modern-day RDBMSs that are widely used include Oracle, MySQL, etc. Oracle is the largest commercially available RDBMS, and MySQL (earlier acquired by Sun, and subsequently by Oracle) is a free and open source RDBMS that is very well-known.

Some other (important) concepts

A database is accessed, read from and written to, using an interface that allows applications to store and retrieve data. This interface is called an *interface driver* or *database driver*. There is usually more than one driver for an RDBMS. ODBC (Open DataBase Connectivity) is a popular database driver. So, if you write a program that needs to store and read data, you would use the API (Application Programming Interface—a collection of functions) provided by ODBC to do so. Similarly, there is JDBC (Java DataBase Connectivity) and others.

Let's suppose you are making a payment online, and the power goes out at the least opportune moment, say, just when you hit the 'Pay' button (as we all know, Murphy's Law never fails us; it's like gravity, I guess). Did the money get deducted from your bank account? If it did, did the merchant receive it? If so, does the system know that it happened successfully?

There are so many steps involved after you push that 'Pay' button on Amazon, eBay or any e-commerce site; like the various messages exchanged between the participating banks, the merchant and the buyer (you). Once the power comes back, you can see what happened: the money has gone from your account, but the order status is 'Unpaid'. The seller says he didn't receive the money. This is where the concept of a transaction comes in. If one step in the multitude of steps fails, then the action won't happen. It's an all-or-nothing scenario. Either your purchase will be complete, with all steps successful, or nothing happens. There is no in-between state. So, in our example, the bank will roll back the changes, and the money will re-appear (depending, of course, on the bank's time policies).

It's about ACID

ACID stands for atomicity, consistency, isolation and durability. We have just seen what transactions are all about. The concept of ACID is nothing but a set of properties that define a successful execution of a transaction. The all-or-nothing attribute of the transaction is called *atomicity*. The data must be valid for the transaction to be successful; this is the *consistency* of the transaction. In the above example, it's a guaranteed possibility that there will be many buyers making purchases on the site. The RDBMS must take care that each transaction is isolated from the rest, to prevent data with errors. This is the *isolation* of the transaction. And finally, we have *durability*, which is an assurance that the RDBMS makes that when the transaction completes successfully, it stays that way—irrespective of any form of failure that may occur once the transaction completes. It's

also a way to make sure that the transaction is rolled back if it has failed intermediately.

Getting in touch with the data: SQL

The theory part is fundamentally in place now. But how does one actually communicate with an RDBMS? This is done by what's called SQL (Structured Query Language; pronounced 'sequel'). SQL is pretty easy to understand, since it's like broken English. I'll give you an example: There is a table that stores the names, addresses, cities, countries and phone numbers of college students. I want the RDBMS to show me all students who are from, say, Mumbai. This would be the SQL statement for that query:

```
SELECT * FROM STUDENT_MASTER WHERE city='Mumbai';
```

Here, the asterisk (*) is synonymous to all—it's a wildcard character. The query will return all rows where the city is 'Mumbai'. This SQL statement can be passed on to the RDBMS via an API function call in code, or through an application like phpMyAdmin. Upon receiving the query, the system returns the records as per the query. Suppose we want only names of the students from Mumbai, and not whole records (we don't want their addresses, phone numbers or countries of origin), the query would then become:

```
SELECT name FROM STUDENT_MASTER WHERE city='Mumbai';
```

Note that the word 'Mumbai' is in quotes because it is text. Numbers won't have quotes in a query. And the statement must end in a semi-colon.

For the next part of this article, I will be using MySQL to create the database. Here's a quick way to go about installing and using it:

Windows users: Download MySQL from its website (it's called the MySQL Community Server). Just run the installer, and you should be okay.

Linux users: Use the package manager and install *mysql*; alternatively, you can build it from source, if you wish to.

You can start MySQL through the command prompt (Windows) or terminal (Linux), by running *mysql -u root -p*. It will ask you for a password; keep it blank since the default is blank. You can change it later.

Types of SQL statements

SQL statements can be classified into the following four categories:

- **DDL—Data Definition Language:** This class of statement is used to create/destroy or define/change database components. Examples: *CREATE*, *DROP*, *RENAME*.
- **DML—Data Manipulation Language:** As the name suggests, these statements manipulate the data itself, and the views related to it. Examples: *SELECT*, *INSERT*.